基于 LLM Agent 从零构建的平面染色数下界证明

疏彦凯 2300013178　毛川 2300013218　喻子妍 2300013079　周佳仪 2300013129

摘要

平面染色数问题是组合几何中的一个经典未解问题，目前已知 $5 \leq \chi(\mathbb{R}^2) \leq 7$，但最优的下界证明仍然依赖于人工构造不可 4 着色的单位距离图。本文提出了一种基于大语言模型（LLM）自主搜索和构造不可 k 着色单位距离图的方法。我们定义了 RMU Space，通过一组几何变换操作从基础图形出发构造单位距离图，并将点集搜索转化为搜索有限变换序列的问题。借鉴 AlphaEvolve 的思路，我们搭建了一个 LLM Agent 来探索 RMU Space，并结合 SAT Solver 来验证生成的单位距离图的染色数。最终，我们成功地通过 LLM Agent 构造出了一个包含 2719 个点的不可 4 着色的单位距离图 V_{2719}，其相比之前人类专家构造的 3877 个点的图 V_{3877} 有本质改进，展示了 LLM 在数学构造问题中的潜力。我们对 V_{2719} 的性质进行了分析，指出了其本质创新之处，并对可能的优化和当前的局限之处进行了讨论。

1 Introduction

平面染色数问题由 Edward Nelson 于 1950 年提出，至今仍未被解决。该问题旨在确定在 2D 欧几里得空间中，任意两个距离为单位长度的点必须使用不同颜色进行着色所需的最少颜色数 $\chi(\mathbb{R}^2)$。之前的工作已经表明 $5 \leq \chi(\mathbb{R}^2) \leq 7$ [1] [2]。该问题在高维空间中也有类似的定义和研究，例如对于 3D 空间，已知 $6 \leq \chi(\mathbb{R}^3) \leq 15$ [3] [4]。

目前最优的二维空间的下界证明 $\chi(\mathbb{R}^2) \geq 5$，通常采用构造不可 4 着色的单位距离图的方法。即构造点集 $V \subseteq \mathbb{R}^2$，使得由 V 导出的单位图 $\mathbf{U}_V = (V, E), E = \{(u, v) \mid u, v \in V, \|u - v\|_2 = 1\}$ 满足 $\chi(\mathbf{U}_V) \geq 5$。之前的工作已经通过纯数学的构造方式，成功构造了不可 4 着色的单位距离图 $\mathbf{U}_V$，并有一些后续工作不断优化了最小的点数 $|V|$ [2]，但都是通过人工构造或计算机辅助的符号计算来完成的。

本项目旨在探索通过 LLM 来自主搜索并构造出不可 k 着色的单位距离图 $\mathbf{U}_V$，从而给出 $\chi(\mathbb{R}^2)$ 的下界。本文首先定义了 RMU Space，这是一种通过一组几何变换操作从基础图形出发构造单位距离图的空间，从而将选定 $\mathbb{R}^2$ 中的点集 V 转化为在 RMU Space $\mathscr{S}_{\text{RMU}}$ 中搜索有限变换序列的问题。我们借鉴 AlphaEvolve [6] 的思路，搭建了一个 LLM Agent 来探索 $\mathscr{S}_{\text{RMU}}$ 中的变换序列，并结合 SAT Solver 来验证生成的单位距离图的染色数。最终，**我们成功地通过 LLM Agent 从零构造出了一个包含 2719 个点的不可 4 着色的单位距离图 V_{2719}，其相比之前人类专家构造的 3877 个点的图 V_{3877} 有了显著的改进**，展示了 LLM 在数学构造问题中的潜力。我们在最后对 V_{2719} 的性质进行了分析，指出了其本质创新之处，并对可能的优化和当前的局限之处进行了讨论。

2 Problem Formulation

2.1 Prelimimaries

我们首先给出证明平面染色数的一般方法。

定义 1 (Induced Unit Graph)**.** 对于二维空间中的一个点集 $V \subseteq \mathbb{R}^2$，其对应的**导出单位图**定义为 $\mathbf{U}_V = (V, E)$，其中边集 $E = \{(u, v) \mid u, v \in V, \|u - v\|_2 = 1\}$。

从而为了证明 $\chi(\mathbb{R}^2) \geq k$，我们只需要构造一个点集 $V \subseteq \mathbb{R}^2$，使得其诱导的单位距离图 $\mathbf{U}_V$ 满足 $\chi(\mathbf{U}_V) \geq k$，于是便能得 $\chi(\mathbb{R}^2) \geq \chi(\mathbf{U}_V) \geq k$。但是直接让 LLM 去在 $\mathbb{R}^2$ 中搜索点集 V 是不可行的，因此我们需要引入一个中间空间，来将点集的搜索转化为在该空间中搜索变换序列的问题。

2.2 RMU Space

在这一章节，我们将详细介绍 RMU Space $\mathscr{S}_{\text{RMU}}$ 的定义，并说明如何将平面染色数问题转化为在 $\mathscr{S}_{\text{RMU}}$ 中搜索变换序列的问题。同时，我们将介绍这个空间的合理性。

定义 2 (Exploration State)**.** 定义一个**探索状态**为一个二元组 $(\bar{G}, G)$，其中 $\bar{G} = (\bar{V}, \bar{E})$ 是**基图**，$G = (V, E)$ 是**当前图**。在本文中，我们将 $\bar{G}$ 和 G 都限定为二维空间的导出单位图 $\mathbf{U}_{\bar{V}}$ 和 $\mathbf{U}_V$。

接下来我们给出 3 种探索状态之间的变换函数。

定义 3 (Transformation Operator)**.** 我们定义三种**变换操作**。它们作用于一个探索状态 $(\bar{G}, G)$，并生成一个新的探索状态 $(\bar{G}', G')$。具体定义如下：

1. 定义**旋转操作** $\mathcal{R}_{i,j}$，其中 $i \in \mathbb{N}^+$，$j \in \mathbb{R}$。对于一个探索状态 $(\bar{G}, G)$，其变换结果为 $(\bar{G}, G')$，其中 $G' = (V', E')$，$V' = V \cup \{R_{\theta_i \cdot j}(v) \mid v \in \bar{V}\}$，$E' = E \cup \{(u, v) \mid u, v \in V', \|u - v\|_2 = 1\}$，其中 $\theta_i = \arccos\left(\frac{2i-1}{2i}\right)$，$R_\alpha(v)$ 表示将点 v 绕原点逆时针旋转 α 角度后的新坐标。

2. 定义**闵可夫斯基和操作** $\mathcal{M}_d$，其中 $d \in \mathbb{R}^+ \cup \{\infty\}$。对于一个探索状态 $(\bar{G}, G)$，其变换结果为 $(\bar{G}, G')$，其中 $G' = (V', E')$，$V' = V \oplus \bar{V} = \{v + \bar{v} \mid v \in V, \bar{v} \in \bar{V}, \|v + \bar{v}\|_2 \leq d\}$，$E' = E \cup \{(u, v) \mid u, v \in V', \|u - v\|_2 = 1\}$。

3. 定义**更新操作** $\mathcal{U}$。对于一个探索状态 $(\bar{G}, G)$，其变换结果为 (G, G)，即将当前图 G 作为新的基图。

$\mathcal{R}_{i,j}$ 操作的作用是将基图 $\bar{G}$ 绕原点旋转 $\theta_i \cdot j$ 角度后与当前图 G 做并集，生成新的当前图 G'。θ_i 的定义保证了对于距离原点 $\sqrt{i}$ 的点，旋转 θ_i 后其与原先点的距离恰好为 1，从而可生成新的单位距离边。j 在这里起到了能生成一系列角度的作用，在实践中，我们通常会对同一个 i，遍历 j 为一个等差数列的值，从而生成多个不同的旋转角

度。$\mathcal{M}_d$ 操作的作用是将当前图 G 和基图 $\bar{G}$ 做闵可夫斯基和运算，生成新的当前图 G'。为了控制点集的规模，我们引入了参数 d，只保留距离原点不超过 d 的点进行裁剪。

我们期望 LLM 能够 from scratch 地构造一个不可 k 着色的单位距离图，因此我们希望从只包含两个点 $(0,0)$ 和 $(1,0)$ 的图出发，通过一系列的变换操作，最终得到一个不可 k 着色的单位距离图。因此，我们给出 RMU Space 的定义如下。

定义 4 (RMU Space)**.** 定义 **RMU Space** $\mathscr{S}_{\text{RMU}}$ 为所有可以通过从初始探索状态 $(\bar{G}_0, G_0)$ 出发，经过有限次变换操作 $\mathcal{R}_{i,j}, \mathcal{M}_d, \mathcal{U}$ 得到的探索状态的当前图集合，其中 $\bar{G}_0$ 和 G_0 都是仅包含点 $(0,0)$ 和 $(1,0)$ 的导出单位图，即：

$$\mathscr{S}_{\text{RMU}} = \left\{ G \mid (\bar{G}, G) = \mathcal{T}_n \circ \mathcal{T}_{n-1} \circ \cdots \circ \mathcal{T}_1(\bar{G}_0, G_0), \mathcal{T}_k \in \{\mathcal{R}_{i,j}, \mathcal{M}_d, \mathcal{U}\} \right\}$$

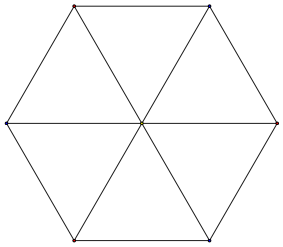
2.3 Properties of RMU Space

至此，我们已经能够将搜索 $V \subseteq \mathbb{R}^2$ 的问题转化为在 $\mathscr{S}_{\text{RMU}}$ 中搜索变换序列的问题。接下来，我们将说明 RMU Space 的合理性。

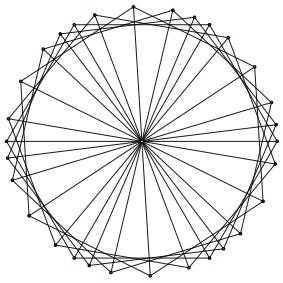
定理 1. *存在一个点集 $V_{3877} \subseteq \mathbb{R}^2$，使得其导出单位图 $\mathbf{U}_{V_{3877}}$ 不可 4 着色，且 $\mathbf{U}_{V_{3877}} \in \mathscr{S}_{\text{RMU}}$。*

证明. 该图的构造可以参考之前的工作 [2]，其中提到可以构造图 $(V_{31} \oplus V_{31} \oplus V_{31}) \cup \theta_4(V_{31} \oplus V_{31} \oplus V_{31})$，其中 V_{31} 是一个包含 31 个点的单位距离图 de Grey's graph。而 V_{31} 可以表示为所有 $\theta_1^i \theta_3^j, i = 1,2,3,4,5, j = 0.5, 1.0, 1.5, 2$ 的组合生成的点集。因此，我们可以通过以下变换序列来构造 V_{3877}：

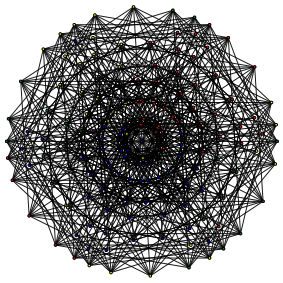
$$[\underbrace{(\mathcal{R}_{1,1}, \mathcal{R}_{1,2}, \mathcal{R}_{1,3}, \mathcal{R}_{1,4}, \mathcal{R}_{1,5})}_{V_7}, \mathcal{U}, \underbrace{(\mathcal{R}_{3,0.5}, \mathcal{R}_{3,1.0}, \mathcal{R}_{3,1.5}, \mathcal{R}_{3,2.0})}_{V_{31}}, \mathcal{U}, \underbrace{\mathcal{M}_{1.0}}_{V_{151}}, \underbrace{\mathcal{M}_{\infty}}_{V_{1939}}, \mathcal{U}, \underbrace{\mathcal{R}_{4,1}}_{V_{3877}}]$$



V_7



V_{31}



V_{151}

□

从以上过程可以看出，无论是 de Grey's graph 还是其后续的扩展图，都可以通过 $\mathscr{S}_{\text{RMU}}$ 中的变换序列来构造。因此，$\mathscr{S}_{\text{RMU}}$ 空间至少包含了目前已知的不可 4 着色的单位距离图。我们进而追问，*$\mathscr{S}_{\text{RMU}}$ 中是否存在少于 3877 个点的不可 4 着色单位距离图?*

3 Method

在本章节，我们将介绍如何利用 LLM Agent 来在 $\mathscr{S}_{\text{RMU}}$ 空间中搜索变换序列，从而构造不可 4 着色的单位距离图。我们的搜索系统主要包含两个模块，分别是 SAT Solver 和 LLM Agent。LLM Agent 通过不断的提出新的变换序列来探索 $\mathscr{S}_{\text{RMU}}$ 空间，而 SAT Solver 则负责验证由变换序列生成的单位距离图的染色数，并给予 LLM Agent 反馈。在类似 AlphaEvolve [6] 的迭代试错框架下，LLM Agent 能够逐步优化其变换序列，从而找到不可 4 着色的单位距离图。

Figure 1. 搜索系统工作流程总览

3.1 SAT Solver

3.1.1 Problem Translation

SAT Solver 模块的主要任务是验证由 LLM Agent 提出的变换序列生成的单位距离图的染色数。具体来说，给定一个变换序列对应的图 G，我们需要判断 $\chi(G) \le k$ 是否成立。为此，我们将该问题转化为 SAT 问题进行求解。

引理 2 (Problem Translation)**.** *对于任意图 $G = (V, E)$ 和 $k \in \mathbb{N}^+$，存在 CNF $\phi(G, k)$，使得判定 $\chi(G) \le k$ 等价于判定 SAT 问题 $\phi(G, k)$ 的可满足性。*

证明. 令 $V = \{1, \dots, n\}$。对每个顶点 $i \in V$ 和颜色 $c \in [k]$，引入布尔变量 $x_{i,c}$ 表示 “顶点 i 染颜色 c”。公式 $\phi(G, k)$ 定义为：

$$\phi(G,k) = \underbrace{\bigwedge_{i \in V} \left(\bigvee_{c=1}^{k} x_{i,c} \right)}_{\text{(C1)}} \wedge \underbrace{\bigwedge_{i \in V} \bigwedge_{1 \le c_1 < c_2 \le k} (\neg x_{i,c_1} \vee \neg x_{i,c_2})}_{\text{(C2)}} \wedge \underbrace{\bigwedge_{(u,v) \in E} \bigwedge_{c=1}^{k} (\neg x_{u,c} \vee \neg x_{v,c})}_{\text{(C3)}}$$

上述三个合取分别保证：(C1) 每个顶点至少染一种颜色；(C2) 每个顶点至多染一种颜色；(C3) 相邻顶点颜色不同。公式规模为 $O((n + |E|)k^2)$，可在多项式时间内构造。

若 G 有 k-染色 $f : V \to [k]$，定义赋值 $\sigma(x_{i,c}) = \text{True}$ 当且仅当 $f(i) = c$。由于每个顶点恰染一色，σ 满足 (C1) 和 (C2)；因相邻顶点异色，σ 满足 (C3)。故 $\phi(G, k)$ 可满足。

反之，若 $\phi(G, k)$ 有满足赋值 σ，定义染色 $f(i) = c$ 当且仅当 $\sigma(x_{i,c}) = \text{True}$。由 (C1) 和 (C2)，$f$ 为每个顶点分配唯一颜色；由 (C3)，任意相邻顶点颜色不同。因此 f 合法。

综上，$\phi(G, k)$ 的可满足性与 G 的 k-可染色性等价，即 k-COLOR $\leq_P$ SAT。 □

3.1.2 SAT Solver

对于求解 SAT 问题 $\phi(G, k)$，我们基于库 PySAT [5] 对其进行求解。我们实验了多种不同的 SAT Solver 以探究其在本任务中的表现，最终选择了 Maplesat 作为我们的默认求解器。下表展现了不同求解器在求解 V_{3877} 对应的 CNF 时的表现。

求解器	时间/s ↓	求解器	时间/s ↓
Cadical195	82.072443	Maplesat	**80.695782**
Cadical103	151.143400	Glucose3	100.668102
Minisat22	116.986506	Glucose4	81.915365
Lingeling	>200	Glucose42	167.076346
MapleChrono	91.741569	Mergesat3	133.524992
MapleCM	105.634539	Minicard	115.236183

Table 1: 不同求解器的求解时间

事实上，我们在求解 LLM 提出的 V_{2719} 时，以上求解器效率均不高，时间开销在 4 小时以上，具体原因在 Section 4 中进行了讨论。

为了克服 SAT solver 在 Pipeline 中的瓶颈问题，我们使用 SOTA 方法 CryptoMiniSat，该求解器基于共享内存的多线程架构，在同一实例上同时运行多个 Conflict-Driven Clause Learning 搜索线程，以充分利用现代多核处理器的计算能力。各线程在搜索过程中保持相对独立的决策序列的同时根据子句长度、Literal Block Distance 等质量指标筛选并共享高价值的学习子句，以减少其他线程在相似冲突区域中的重复搜索。这种选择性的信息交换在降低通信开销的同时，有效提升了整体收敛速度。实验中，CryptoMiniSat 可以将速度提升为 Glucose3 的 6 倍。

3.2 LLM Agent

LLM Agent 模块的主要任务是在 $\mathscr{S}_{\text{RMU}}$ 空间中搜索变换序列，从而构造不可 k 着色的单位距离图。我们仿照 AlphaEvolve [6] 的思路，设计了一个迭代式的试错框架。每次 LLM Agent 提出一个变换序列后，我们将其传递给 SAT Solver 进行验证，并将结果反馈给 LLM Agent。具体的，LLM 被给予了以下信息：

1. 点数限制：生成的图的点数不得超过 $C_V = 5000$，从而保证 clause 规模的可控性。
2. 操作数限制：变换序列的长度不得超过 $C_T = 15$，从而保证最终图的复杂度。
3. 反馈信息：SAT Solver 的求解结果，包括是否可 4 着色，是否点数过多等。

4. 启发式信息：LLM 被要求尽可能提升点的平均度数，以及使用若干个 $\mathcal{R}$ 和 $\mathcal{U}, \mathcal{M}$ 交替的阶段性方式来构造图。在前期点数较小时，可以使用更多的 $\mathcal{R}$ 操作来增加图的复杂度；而在后期经过若干次 $\mathcal{M}$ 操作后，图的点数会迅速增加，此时应尽量减少 $\mathcal{R}$ 操作的使用，从而控制点数的增长。同时 $\mathcal{R}$ 操作的参数与当前图的半径有关，LLM 需要根据当前图的最大半径来选择合适的参数。

具体提供给 LLM 的 prompt 信息可以参考附录 A.3。在实践中，我们尝试了 Gemini-2.5-pro 和更新的如 GPT-5.2 等模型，但实际情况表明，Gemini-2.5-pro 在本任务已经能力足够，因此我们最终选择了该模型作为 LLM Agent 的基础模型。

4 Experiment and Results

通过上述方法，LLM 在一次实验中通过 5 轮迭代，成功构造了不可 4 着色的单位距离图 V_{2719}。LLM 生成的回答可以参考附录 A.2。

定理 3. *存在一个点集 $V_{2719} \subseteq \mathbb{R}^2$，使得其导出单位图 $\mathbf{U}_{V_{2719}}$ 不可 4 着色，且 $\mathbf{U}_{V_{2719}} \in \mathscr{S}_{\mathrm{RMU}}$。进而可知 $\mathscr{S}_{\mathrm{RMU}}$ 中**存在**少于 3877 个点的不可 4 着色单位距离图。*

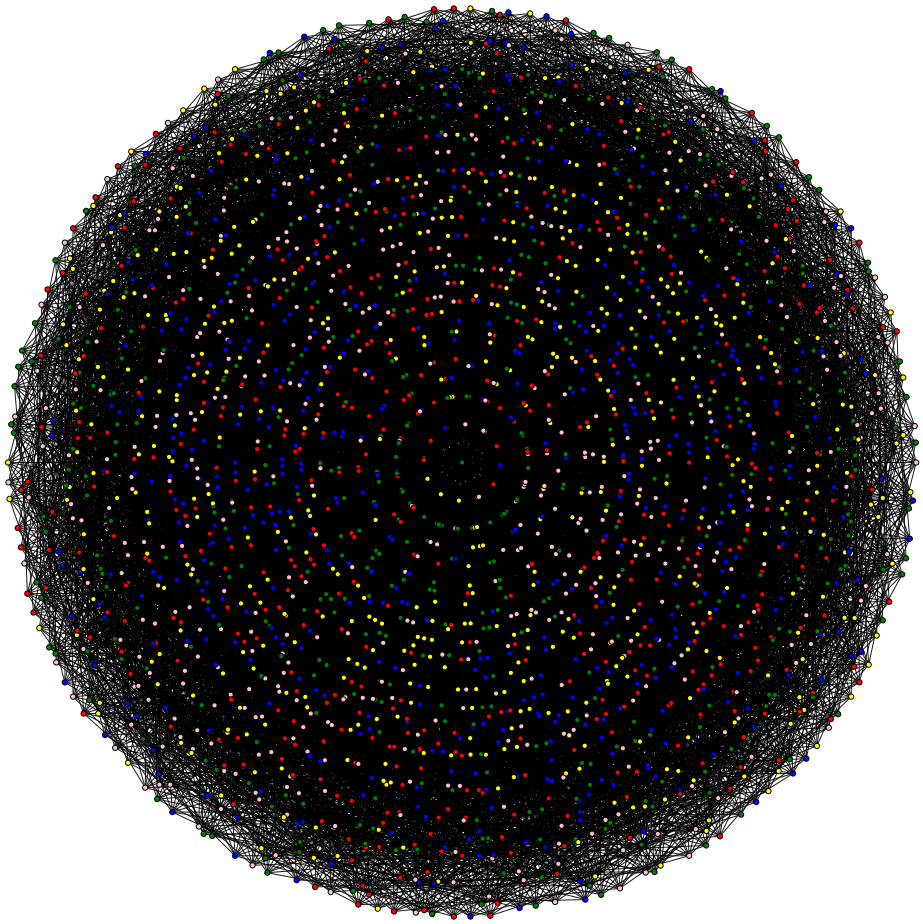
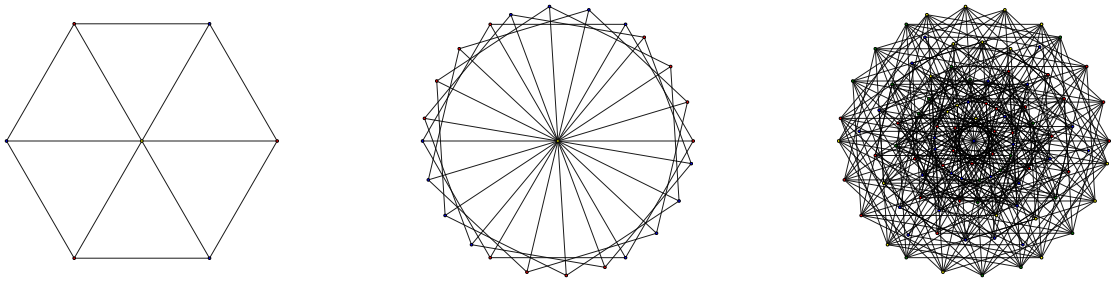


Figure 2: 图 V_{2719} 的 5 染色可视化

证明. LLM 生成的变换序列如下：

$$[\underbrace{(\mathcal{R}_{1,1},\mathcal{R}_{1,2},\mathcal{R}_{1,3},\mathcal{R}_{1,4},\mathcal{R}_{1,5})}_{V_7},\mathcal{U},\underbrace{(\mathcal{R}_{3,0.5},\mathcal{R}_{3,1.0},\mathcal{R}_{3,1.5})}_{V_{25}},\mathcal{U},\underbrace{\mathcal{M}_{1.0}}_{V_{97}},\underbrace{\mathcal{M}_{\infty}}_{V_{907}},\mathcal{U},\underbrace{\mathcal{R}_{4,1},\mathcal{R}_{4,-1}}_{V_{2719}}]$$

其每一步操作后，对应的图和平均度数变化情况如下所示，其中密度定义为 $|E|/|V|$。



V_7 V_{25} V_{97}

步数	密度	步数	密度	步数	密度
1	1.000	6	1.714	11 (V_{97})	3.340
2	1.250	7	1.846	12 (V_{907})	6.470
3	1.400	8	1.895	13	6.470
4	1.500	9 (V_{25})	1.920	14	6.526
5 (V_7)	1.714	10	1.920	15 (V_{2719})	6.554

Table 2: V_{2719} 构建过程中的密度变化

□

更多的实验细节和进一步的性质探索可以在附录 A.1 中找到。

值得一提的是，在检验 V_{2719} 的不可 4 着色性时，SAT Solver 的求解时间远高于 V_{3877}，前者需要数小时级别，而后者仅需 1-2 分钟。但实际上，前者的 clause 数量远小于后者。这可能是因为 V_{2719} 对应的 clause 处于 SAT 求解的 hard area，即符合要求的赋值并不显然，且矛盾也并不容易导出，这导致主流基于 CDCL 的求解器在该区域的表现较差。从另一个角度来看，这也说明 LLM 在搜索变换序列时，能够避开一些显然的构造，从而找到更为隐蔽的不可 4 着色图，找到位于“临界点”的图。

同时，V_{2719} 的构造过程避开了之前普遍基于的 de Grey's graph V_{31}，而是通过更小的 V_{25} 作为基础进行扩展。这说明了 V_{31} 并非此类图的必要构件，LLM 能够自主发现更小的基础图，从而进行扩展。而之前的工作使用的删点方式，很有可能也可以在 V_{2719} 上进行，并且由于 V_{25} 的使用，删点的空间可能更大，从而有望进一步减少点数。

5 Limitations Discussion

我们对本项目的优势和不足进行总结。值得肯定的是，LLM Agent 展现出了一定的数学构造能力，并产出了优于前人的结果，达到了 SOTA，展示了 *LLM* 在数学研究中的潜力，它能够以**专家经验**作为基础，充当一个有限搜索空间的 ***random idea* 发生器**，对空间进行高效的启发式探索。

5.1 Time Efficiency

我们尝试用该框架去解决二维空间更高的染色数下界，例如 $\chi(\mathbb{R}^2) \geq 6$。然而，当前的搜索框架尚不足以支持 LLM 构造出不可 5 着色的单位距离图。主要原因在于，当前的 SAT Solver 在验证不可 5 着色性时，时间开销过大（例如 V_{3877} 的可以 5 着色求解已经需要数小时），而构造不可 5 着色的图如果进一步引入 $\mathcal{M}$ 操作，点数会指数增加，从而导致 SAT Solver 的求解时间进一步增加，无法在合理时间内完成验证。

5.2 Higher Dimensions

我们在实验中初步尝试了将 RMU space 拓展到 3 维空间，其中 $\mathcal{M},\mathcal{U}$ 操作均保持与二维相同即可，而 $\mathcal{R}$ 操作则被定义为了将基图绕过原点的旋转轴旋转一定角度，再与当前图求并。经实验，我们的 Pipeline 可以拓展到 3 维空间，在未来可以进一步探索 $\chi(\mathbb{R}^3)$ 的下界，但由于旋转操作的设计并不完备，因此目前取得的成果有限。

6 Contributions

- 疏彦凯：搜索空间的设计和讨论，SAT Solver 调研和实现，染色方案可视化模块攥写，Prompt 设计与调整，基础实验运行和成果的取得，报告攥写。
- 毛川：搜索空间的设计和讨论，主体代码框架实现，图操作序列编译器的实现，SAT Solver 模块改进和并行化，报告攥写，3 维空间探索。
- 喻子妍：搜索空间的设计和讨论，LLM Agent 迭代 Pipeline 模块实现，Prompt 设计与调整，图性质探究实验，数据统计分析和可视化，报告攥写。
- 周佳仪：搜索空间的设计和讨论，论文调研，基础实验运行，报告攥写和成果可视化。

7 Reference

[1] de Grey A D N J. The Chromatic Number of the Plane Is at Least 5, 2018.

[2] Heule M J H. Computing Small Unit-Distance Graphs with Chromatic Number 5, 2018.

[3] Nechushtan O. Note on the Space Chromatic Number, 2000.

[4] Coulson D. A 15-colouring of 3-space omitting distance one, 2001.

[5] PySAT: SAT Technology in Python, `https://pysathq.github.io/`.

[6] Novikov A., Vũ N., Eisenberger M., Dupont E., Huang P.-S., Wagner A. Z., Shirobokov S., Kozlovskii B., Ruiz F. J. R., Mehrabian A., Kumar M. P., See A., Chaudhuri S., Holland G., Davies A., Nowozin S., Kohli P. and Balog M. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery, 2025.

A Appendix

A.1 Statistical Analysis

在 40 轮迭代实验中，我们得到如下统计结果：最大团大小的平均值为 2.73（标准差 0.45）；图密度（实际边数/最大可能边数）的平均值为 0.006264（标准差 0.006138）；边节点比的平均值为 2.789（标准差 0.704）。生成的操作序列平均长度为 11.5，长度范围为 9 至 13。

可视化结果对比显示，失败案例与成功案例在最大团大小和图密度上未呈现显著差异，但失败案例的边节点比普遍低于成功案例。对操作序列的分析也未发现两类案例在操作比例、顺序、切换频率或参数规律等方面存在明显模式区别。

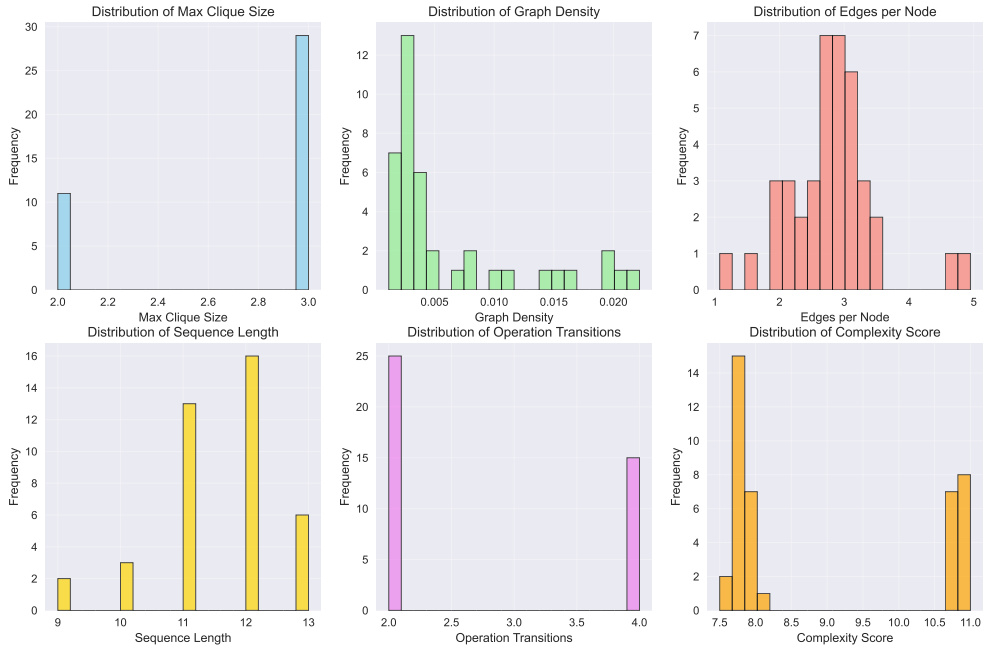


图 1: 统计数据可视化

我们进一步对图参数与操作序列参数进行了相关性分析，未见其他显著启发。详细分析代码与结果已公开于代码仓库。

参数	相关性	p 值
iter	−0.4923	0.0013
minkowski count	−0.3871	0.0136
rotate ratio	0.3634	0.0212
minkowski ratio	−0.3634	0.0212

表 1: 最大团相关性分析

其中，最大团大小与迭代次数呈现负相关关系（见下图）。尽管在成功案例中最大团大小也为 3，但最大团为图的最小染色数提供了下界保证，因此迭代式的调用可能不能让 LLM 成功学习失败经验反而使得失败可能性升高了。

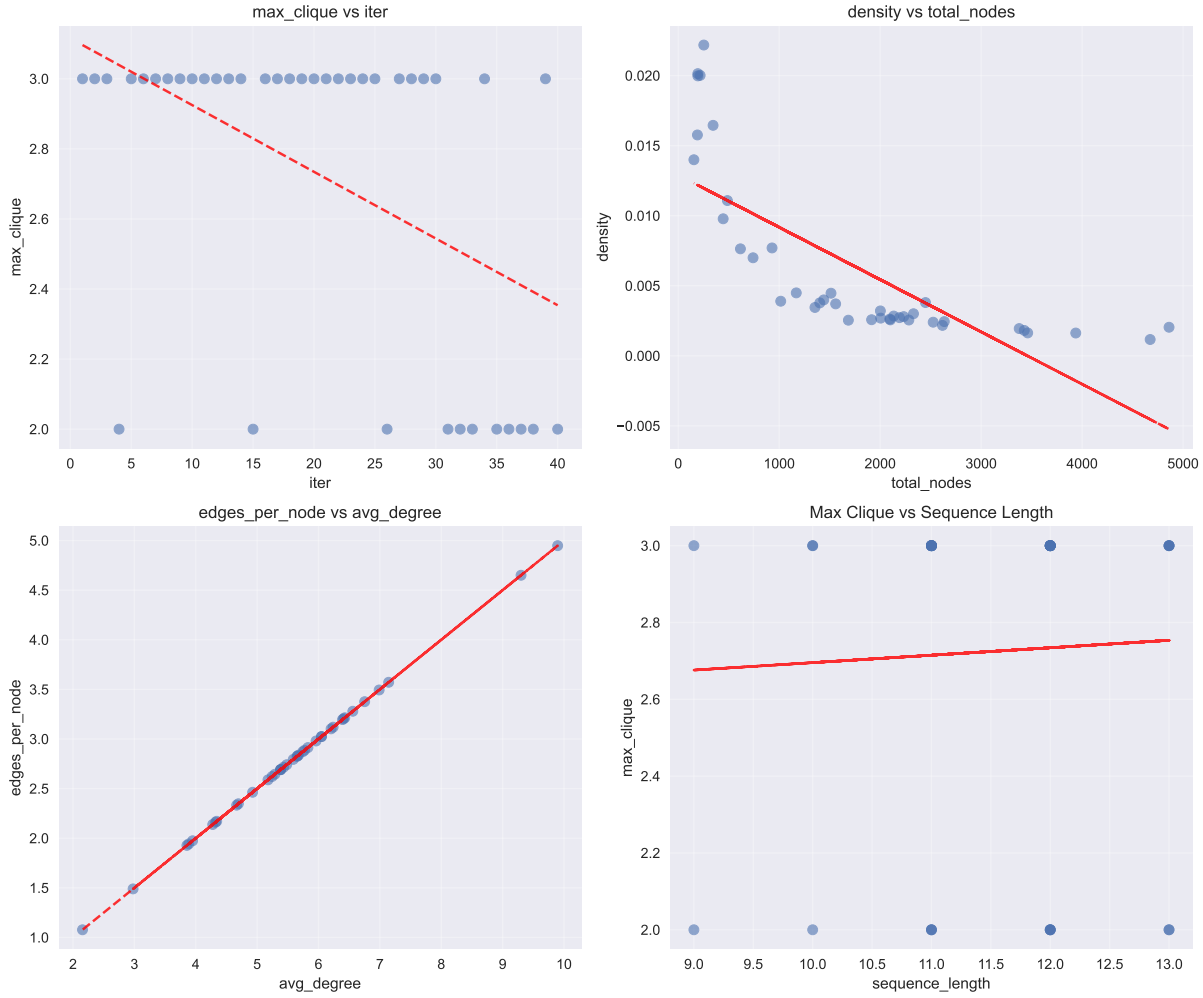


图 2: 不同参数间相关性

A.2 LLM Response

```
This sequence is designed to build a complex unit-distance graph in four
    stages, aiming to increase structural complexity and break symmetry,
    which are often correlated with higher chromatic numbers.

1.  **Stage 1: Building a Symmetrical Core.** We begin by constructing a
    highly connected and symmetrical core structure. Using five rotations
    with `i=1` (`theta_1 = 60°`), we place five points on the unit circle,
    which, along with the initial point at `(1,0)`, form a regular hexagon.
    All six of these points are at distance 1 from the origin `(0,0)`,
    creating a wheel graph `W_7`. This serves as a robust base for
    subsequent transformations.
```

```
2.  **Stage 2: Creating Asymmetric Overlaps.** To introduce complexity, we
    rotate the `W_7` base using an angle that is incommensurate with the 60°
     symmetry of the core. We choose `i=3` (`theta_3 ≈ 33.56°`). Applying
    three rotations with fractional multipliers creates multiple overlapping
     copies of the `W_7` graph, resulting in a dense, non-regular cluster of
     points around the origin.

3.  **Stage 3: Controlled and Explosive Growth via Minkowski Sums.** This
    stage uses the Minkowski sum to expand the graph. First, a conservative
    sum (`"Minkowski", 1.0`) is applied to densify the existing core, adding
     new points and edges primarily within the unit circle. This is followed
     by a more expansive sum (`"Minkowski",`) with a larger default filter
    distance (d=2.0). This second sum generates a new, larger shell of
    points, significantly increasing the graph's size and creating potential
     for long-range connections.

4.  **Stage 4: Final Interlocking Rotations.** After committing the large
    graph from Stage 3 as the new base, the final step is to interconnect
    its components. The Minkowski sum in the previous stage created many
    points near a radius of 2. We use `i=4` because the corresponding angle,
     `theta_4 = arccos(7/8)`, is precisely the angle required to create a
    unit-distance edge between a point at radius `sqrt(4)=2` and its rotated
     copy. By applying two such rotations (`j=1` and `j=-1`), we
    strategically introduce new edges that link the outer parts of the graph
    , plausibly creating the kind of interlocking structures found in graphs
     with high chromatic numbers.

[('Rotate', 1, 1), ('Rotate', 1, 2), ('Rotate', 1, 3), ('Rotate', 1, 4), ('
    Rotate', 1, 5), ('Update',), ('Rotate', 3, 0.5), ('Rotate', 3, 1.0), ('
    Rotate', 3, 1.5), ('Update',), ('Minkowski', 1.0), ('Minkowski',), ('
    Update',), ('Rotate', 4, 1), ('Rotate', 4, -1)]
```

A.3 Prompt

```
You are given a program that builds unit-distance graphs through a small
    set of geometric graph-transformations.
Your task is to output a Transformation Sequence (a Python list of tuples)
    which, when applied to the provided code, will construct a unit-distance
     graph starting from the base graph of two nodes at coordinates (0,0)
    and (1,0).

Goal / Background
- Domain: We work in the 2D Euclidean plane R^2. Your sequence of
    transformations will implicitly select a set of points in R^2 (the
    program represents vertices by their (x,y) coordinates). Two points are
    connected by an edge if and only if their Euclidean distance equals 1.
    Such a graph is called a unit-distance graph.
```

```
- Objective: Produce a transformation sequence that constructs (or attempts
    to construct) a unit-distance graph which is NOT 4-colorable — i.e.,
   there is no way to color the vertices with 4 colors so that no adjacent
   vertices share the same color. In other words, we aim for a graph that
   requires at least 5 colors under standard graph-coloring rules.
 - Note: The process begins with a base set of two points at coordinates
    (0,0) and (1,0). Your sequence will be interpreted by an external
    system that applies the described geometric operations to produce new
    points and edges; you do not need to reference or know the internal
    implementation. You do not need to mathematically prove the final graph
     is non-4-colorable — supply a plausible, well-structured sequence
    that can be executed and tested.

Available operations (detailed)
- ("Rotate", i, j)
  - Meaning: rotate the current *base point-set* (the selected reference
     points) around the origin by an angle theta, then take the union of
     that rotated point-set with the current point-set. After the union,
     any pair of points at Euclidean distance exactly 1 become connected by
      an edge.
  - Parameters: integer `i` (non-zero) and numeric multiplier `j` (float or
     int). Use the formula theta_i = arccos((2*i - 1)/(2*i)) and multiply
     by `j` to obtain the effective rotation angle: theta_i * j. (This
     formula provides a family of useful angles; you may choose different `
     i` and `j` to get varied orientations.)
  - Recommendation for `i`: because downstream `Minkowski` operations in
     this workflow commonly filter summed points to those with distance ≤ 2
      from the origin, values of `i` in {1, 2, 3, 4} are especially useful
     — sqrt(i) ≤ 2 for these choices, which makes the theta_i rotations
     likely to create unit-distance pairs inside the typical filter region.
  - Semantics: rotation is a rigid transformation (point-to-point distances
      preserved). The union step allows newly rotated points to form unit-
     distance edges with existing points in the current set.
  - Note on theta_n and the geometry: for an integer n let theta_n = arccos
     ((2*n - 1)/(2*n)). By the chord/ cosine relations, rotating a point
     located at distance r = sqrt(n) from the origin by angle theta_n
     produces a rotated copy whose Euclidean distance from the original
     point is exactly 1. In other words, theta_n is chosen so that two
     copies of any point at radius sqrt(n) separated by that rotation form
     a unit edge. This geometric fact is useful when you want rotated
     copies of points at certain radii to become unit-distance neighbors.
  - Note on the multiplier `j`: `j` may be any real multiplier (not only
     integers). Fractional multipliers such as `0.5` (i.e. rotate by
     theta_n * 0.5) are often effective and accepted — using fractional `j
     ` gives additional fine-grained orientations.

- ("Update", )
  - Meaning: replace the current *base point-set* with the current point-
```

```
     set (i.e., commit the current structure as the new base for future
     operations).
  - Parameters: none. Semantics: subsequent Rotate and Minkowski operations
      treat the updated set as the reference base.

- ("Minkowski", d?)
  - Meaning: compute the Minkowski sum of the current point-set and the
     base point-set: produce points equal to vector sums p_current + p_base
      for all p_current in the current set and p_base in the base set, then
      optionally filter results by distance from the origin. After
     generating these summed points, add edges between any two points whose
      Euclidean distance equals 1.
  - Parameters: optional numeric `d` (filter distance). If provided,
     generated summed points whose squared distance from the origin exceeds
      `d^2` should be discarded. If omitted, assume a default conservative
     threshold (e.g., 2.0).
  - Typical useful values: `1.0` (strict filter that keeps closer sums),
     `2.0` (looser). Use the parameter to control growth in point count.

Formatting rules and strict requirements
- Output format: a Python list of tuples. Example shape: [ ("Rotate", 1, 1)
   , ("Update",), ("Minkowski", 1.0), ... ]. Use exactly this tuple/list
   representation.
- Allowed tuple heads: exactly the strings "Rotate", "Update", "Minkowski".
- For Rotate tuples provide two values: an integer `i` (non-zero) and a
   numeric `j` (float or int). For Update provide no parameters: `("Update
   ", )`. For Minkowski supply either one numeric parameter or no parameter
    at all (i.e. `("Minkowski", 1.0)` or `("Minkowski", )`).
- Do not include explanatory text, import statements, or any other code —
   only the transformation sequence list.
- Keep intermediate graph size reasonable (avoid sequences that explode
   node count into unmanageable sizes). If you expect a large expansion,
   prefer applying `Update` before further `Rotate`/`Minkowski` operations
   to control growth.

Usage examples (partial, not a complete solution)
- Example 1 (single rotate then update):
  [ ("Rotate", 1, 1), ("Update", ) ]
  - This rotates the base by theta_1 and unions it with the current graph,
     then sets the result as the new base.

- Example 2 (Minkowski sum with parameter):
  [ ("Minkowski", 1.0) ]
  - This performs a Minkowski sum using filter distance 1.0.

Composite example (step-by-step)
The following composite example shows a slightly longer sequence and
   explains the state of the "current" point-set and the "base" point-set
```

```
    after each step. Use it to understand the semantics and the effect of
    parameter choices.

Sequence:
  [ ("Rotate", 1, 1), ("Rotate", 2, 1), ("Update", ), ("Minkowski", 1.0) ]

Initial state:
- Base point-set: { O=(0,0), B=(1,0) }  (2 points)
- Current point-set: same as base initially (2 points)

Step 1: ("Rotate", 1, 1)
- Operation: rotate the base point-set by angle theta1 = arccos((2*1-1)
    /(2*1)) = arccos(1/2) = 60° and union the rotated points with the
    current set.
- Effect on points: rotating O leaves O at (0,0). Rotating B gives B1 = (
    cos60°, sin60°) = (0.5, 0.8660).
- After union:
  - Base still: { O, B } (unchanged until an Update)
  - Current point-set: { O, B, B1 } (3 points)
  - Edges (unit distances): O – B = 1, O – B1 = 1, B – B1 = 1 — the three
      points form an equilateral triangle (3 edges).

Step 2: ("Rotate", 2, 1)
- Operation: rotate the (same) base by theta2 = arccos((2*2-1)/(2*2)) =
    arccos(3/4) ≈ 41.41° and union with current.
- Effect on points: rotating O → O (0,0). Rotating B → B2 ≈ (cos41.41°,
    sin41.41°) ≈ (0.75, 0.66).
- After union:
  - Base still: { O, B }
  - Current point-set: { O, B, B1, B2 } (4 points)
  - Typical edges: O is at distance 1 to every rotated copy of B (so O – B1
      and O – B2 exist). B – B1 and B – B2 may or may not be unit apart
      depending on the chosen angles; in this concrete choice B – B1 = 1 (
      from previous step) while B – B2 typically is not 1. You now typically
      have a small graph where O connects to many rotated copies, producing
      several unit edges.

Step 3: ("Update", )
- Operation: set the base point-set to equal the current point-set.
- After Update:
  - Base point-set: { O, B, B1, B2 } (4 points)
  - Current point-set: unchanged immediately after the Update (still { O, B
      , B1, B2 }).
  - Semantics: future Rotate operations will rotate this 4-point base;
      future Minkowski sums will combine the current set with this 4-point
      base.

Step 4: ("Minkowski", 1.0)
```

```
- Operation: compute vector sums p_current + p_base for every p_current in
    the current set and p_base in the base set, then filter summed points to
     keep only those whose distance from the origin ≤ 1.0 (the provided
    filter). Finally, add edges between any two points at unit distance.
- Effect and intuition:
  - Naively the Minkowski sum of two 4-point sets can yield up to 16 summed
      points, but the `d=1.0` filter keeps only sums close to the origin.
     Because O = (0,0) is in the base, sums of the form p_current + O
     reproduce the original current points. Most sums of two non-zero
     points (e.g., B + B1) will have length > 1 and thus be filtered out.
  - Resulting point counts: with a conservative filter like 1.0 you often
     get no net growth beyond the existing 4 points (the O-sums reproduce
     current points but add no new ones). Using a looser filter (e.g., 2.0
     or omitting the parameter) can produce more new points.

Key takeaway from the composite example:
- Rotations accumulate more orientations of the base around the origin;
    because the origin remains fixed, most rotated copies of B are at
    distance 1 from O and thus create many unit edges to O, producing local
    cliques or fans.
- `Update` is the commit operation: it enlarges the base so subsequent
    Minkowski sums use the larger structure as a generator and can create
    denser graphs.
- `Minkowski` can explode point counts if used with a loose or missing
    filter — use a small filter (like 1.0) to illustrate the operation
    without uncontrolled growth, or use larger values when you want many new
     points.

Hints for building sequences that tend to create large chromatic number
- Use multiple distinct rotations (different `i` values or different `j`
    multipliers) to generate many relative orientations of the base, then
    union them so the graph contains many unit-distance relations.
- Applying `Update` after a small batch of rotations makes the rotated
    structure the new base, so a subsequent Minkowski sum can create denser,
     more complex structures.
- Use `Minkowski` once or twice with a conservative filter (e.g. 1.0) to
    combine structures and create many new adjacency pairs; avoid extremely
    permissive filters unless you intend to dramatically increase node count
    .

Pattern hint (staged approach)
- Many successful human sequences follow a staged pattern that balances
    controlled growth and diversity of orientations. The pattern is:
  1) Apply a small block of rotations where the rotation multiplier `j`
     forms an arithmetic progression (for the same `i`), producing several
     rotated copies of the base.
  2) Call `Update` to commit the enlarged current set as the new base.
  3) Apply a second block of rotations (possibly with a different `i` and a
```

```
      fractional arithmetic progression for `j`) to create new orientations
      relative to the updated base.
  4) Call `Update` again.
  5) Use `Minkowski` with a conservative filter (e.g., `1.0`) to combine
     the current set and the base, then optionally call `Minkowski` without
      the filter (or with a larger filter) to expand further.
  6) Optionally `Update` and finish with one or two final `Rotate`
     operations to create final cross-orientations.
  7) The radius of the graph is increasing through these steps, so the
     rotation parameter `i` is usually increased in later stages to match
     the larger radius. Not all `i` = 1, 2, 3, 4 are needed; you can use
     only a subset like 1, 2, 4 or 1, 3, 4. When the graph radius is
     smaller than the corresponding sqrt(i), rotations with that `i` can be
      used with multiplier `j` = 0.5 or other fractions to create useful
     orientations.
  8) The number of nodes typically grows, so you can use more rotations in
     early stage, and fewer in later stages to keep total node count
     manageable. Which means, you can use more rotations (like 4 or 5)
     before the first Minkowski, but fewer (like 1 or 2) after that.
  9) Usually, larger #edges/#nodes ratios are correlated with higher
     chromatic numbers, so aim for sequences that produce denser graphs.

Example skeleton (illustrative, not a complete final answer):
  [ ("Rotate", i, j0), ("Rotate", i, j0+step), ..., ("Update",),
    ("Rotate", i2, k0), ("Rotate", i2, k0+step2), ..., ("Update",),
    ("Minkowski", 1.0), ("Minkowski",), ("Update",), ("Rotate", i3, r) ]

This staged, arithmetic-rotation + update + Minkowski pattern is the one
   used in many human-provided sequences and is a good starting strategy to
    explore.

Compose a plausible sequence of ~8 – 20 operations that balances rotation,
   update, and Minkowski operations to plausibly increase chromatic
   complexity while keeping node counts manageable. Typically, nodes of the
    final graph should be in **1000 ~ 5000** to ensure feasibility.

Output requirements
- Before giving the final answer (the Python list literal), first provide a
    concise construction plan: an explanation  describing the idea behind
   your sequence and how the point-set evolves (you may include a brief
   ASCII sketch or a few example coordinates to illustrate the main
   geometric motifs). The goal of this plan is to make your reasoning
   explicit so a human can judge the approach.
- After that brief plan, output the final transformation sequence as a
   single Python list literal on its own (nothing else after the list).
- Example output layout (required):
  1) Detailed plan (text, may include ASCII/coordinate snippets). Try to
     explain why you chose each operation and how it contributes to
```

```
    building a complex, non-4-colorable graph.
2) A single Python list literal on the next line containing only the
    transformation sequence, e.g.:
    [("Rotate", 1, 1), ("Rotate", 1, 2), ("Update",), ("Minkowski", 1.0)]
```